

src\case_studies\E_blockbeam\dynamics.py

Generated: 2026-02-26 12:33:39

dynamics.py

src\case_studies\E_blockbeam\dynamics.py

```
# 3rd-party
import numpy as np

# Local (controlbook)
from case_studies.E_blockbeam import eom_generated
from ..common.dynamics_base import DynamicsBase
from . import params as P

class BlockbeamDynamics(DynamicsBase):
    def __init__(self, alpha=0.0):
        super().__init__(
            state0=np.array([P.z0, P.theta0, P.zdot0, P.thetadot0]),
            u_max=P.force_max,
            u_min=-P.force_max,
            dt=P.ts,
        )
        self.m1 = self.randomize_parameter(P.m1, alpha)
        self.m2 = self.randomize_parameter(P.m2, alpha)
        self.length = self.randomize_parameter(P.length, alpha)
        self.g = P.g

    def f(self, x, u):
        # we can either implement the dynamics here directly, or call the
        # generated dynamics function from generate_state_variable_form.py.
        # Here, we'll call the generated function, but show the direct implementation
        # commented out below.

        xdot = eom_generated.calculate_eom(
            x,
            u,
            m1=self.m1,
            m2=self.m2,
            ell=self.length,
            g=self.g,
        )

        # z, theta, zdot, thetadot = x
        # F = u[0]

        # torque = F * self.length * np.cos(theta)
        # friction = 2 * self.m1 * z * zdot * thetadot
        # block_moment = self.m1 * self.g * z * np.cos(theta)
        # beam_moment = self.m2 * self.g * 0.5 * self.length * np.cos(theta)
        # inertia = self.m2 * self.length**2 / 3 + self.m1 * z**2

        # zddot = z * thetadot**2 - self.g * np.sin(theta)
        # thetaddot = (torque - friction - block_moment - beam_moment) / inertia

        # xdot = np.array([zdot, thetadot, zddot, thetaddot])
```

```
        return xdot

    def h(self):
        y = self.state[:2]
        return y
```